

Retail Oracle: Predictive Cart Intelligence

Phase 1 Deliverables: Design & Architecture

1. Project Plan & Task Breakdown

Sprint 1: Discovery & Design

Focus: Understanding the business context, initial data exploration, and laying out the architectural foundation.

- **Business Understanding (Product/Business Analyst)**
 - Define key success metrics and operational business constraints.
 - Identify stakeholders and map business impact.
- **Data Exploration & EDA (Data Scientist)**
 - Analyze `orders.csv`, `products.csv`, `aisles.csv`, `departments.csv`.
 - Understand data sparsity, reorder patterns, and basket size distribution.
- **Data Cleaning (Data Engineer)**
 - Handle anomalies, missing values, and construct the base schema.
- **Feature Brainstorming (Data Scientist / ML Engineer)**
 - Ideate baseline features (user, product, interaction).
- **HLD & LLD Creation (ML Architect)**
 - Draft High-Level and Low-Level Design architectures for the prediction pipeline.
- **Baseline Target Derivation (ML Engineer)**
 - Define the classification target (`reordered_target`) and split strategy.

Deliverables: EDA Report, Core Feature Schema, Baseline Architecture Document.

Dependencies: Instacart dataset access.

Sprint 2: Implementation & Optimization

Focus: Developing the ML pipeline, creating advanced predictive models, and deploying the interactive UI.

- **Advanced Feature Engineering (Data Engineer / ML Engineer)**
 - Construct user-item interaction signals, temporal features, and popularity indexes.
- **Model Comparison (Data Scientist)**

- Implement and tune ≥ 3 models (Logistic Regression, Random Forest, XGBoost).
- **Evaluation & Metric Review (Data Scientist)**
 - Compute Business Metrics (Recall@K, Precision@K, AUC-ROC).
- **Error Analysis (Data Scientist / ML Architect)**
 - Analyze false positives/negatives, feature importance, and handle cold-start boundaries.
- **Streamlit UI Development (Frontend/ML Engineer)**
 - Build an interactive prediction frontend incorporating threshold sliders and Top-K selectors.
- **Final Optimization (ML Engineer)**
 - Optimize memory usage, prediction latency, and inference caching.

Deliverables: Production-ready Pipeline, Trained XGBoost Model, Interactive Streamlit UI, Evaluation Report. **Dependencies:** Completed feature logic from Sprint 1.

2. High-Level Design (HLD)

Business Problem

Customers on the Instacart grocery platform often buy the same items (milk, eggs, fruit) but suffer friction when re-searching for them.

- **Goal:** Predict which historical items a user will add to their current cart.
- **Impact:** Faster checkout, increased cart density (upselling), and improved customer retention.
- **Stakeholders:** E-commerce UI/UX teams, Category Managers, and Data Science leadership.

System Architecture

The system follows a modular, batch-inference architecture layered below a real-time web application.

1. **Raw Data:** Immutable historical event logs mapping users, items, and sequential orders.
2. **Feature Store / Generation:** Scheduled or dynamic generation of User Profiles, Product Popularity, and User-Product synergy matrices.
3. **Model:** Gradient Boosted Decision Tree (XGBoost) designed for binary classification with probability scoring.
4. **API / UI:** Streamlit wrapper orchestrating candidate generation $\rightarrow$ feature hydration $\rightarrow$ model inference $\rightarrow$ Top-K filtering.

Data Flow

1. Raw orders and order_products_prior CSVs are batched.
2. Data logic merges user histories to isolate user-item overlap.
3. Feature engineering aggregates total stats into a flat mathematical matrix.
4. Target labels are generated: 1 if item was in the "next" cart, 0 otherwise.
5. The Model scores candidates between 0.0 and 1.0.
6. UI extracts the N candidates with probabilities $\geq \text{Threshold}$.

Model Strategy (Classification)

Why Binary Classification rather than traditional Collaborative Filtering or Ranking?

- Recommendation engines easily fall prey to "Filter Bubbles." We strictly want to predict *reorders*.
- Classification provides a calibrated mathematically valid **Probability** (0.0 to 1.0) rather than an arbitrary similarity score, allowing the business to set strict confidence thresholds.

Evaluation Metrics

We avoid generic accuracy because we operate on highly imbalanced data (the user won't reorder 99% of the catalog).

- **Recall@K:** (Primary) If the user bought 5 items, how many of those 5 were in our Top-10 recommendations? This drives business value.
- **AUC-ROC:** Measures the model's fundamental ability to rank positive signals higher than negative signals across all thresholds.

Assumptions & Risks

- **Risk (Cold Start):** New users have no prior cart logic. We assume fallback to global popularity metrics.
- **Risk (Data Sparsity):** Some products are bought very rarely, leading to statistical noise.
- **Risk (Scalability):** Generating user-item interaction pairs for 100k users $\times$ 50k products results in memory crashes. We must restrict candidate generation dynamically to items a user has *already bought at least once*.

3. Low-Level Design (LLD)

Feature List

1. User-Level Features:

- `u_total_orders`: Loyalty and stability index.
- `u_avg_basket_size`: Number of products typically bought per trip.
- `u_reorder_ratio`: Likelihood this user buys the same things vs explores.

2. Product-Level Features:

- `p_total_purchases`: Global popularity.
- `p_reorder_prob`: Global conditional probability that if bought once, it gets bought again.

- `p_avg_cart_position`: Is this an anchor item (often added 1st/2nd)?

3. User-Product Interaction Features:

- `up_total_purchases`: Raw count of times User bought Product.
- `up_order_rate`: (Purchases of Product) / (Total User Orders).
- `up_first_order_num` & `up_last_order_num`: Recency tracking to punish stale habits.

Data Preprocessing

- **Candidate Generation Filter**: Only create \$(User, Product)\$ rows if the user has purchased the product ≥ 1 time before. (Massive dimensionality reduction).
- **Missing Value Handling**: Missing continuous parameters default to \$0.0\$.
- **Encoding**: Tree-based models (RF/XGBoost) mitigate need for One-Hot Encoding. User/Product IDs are actively dropped before training to prevent cardinality explosions and overfitting.
- **Scaling**: XGBoost handles unscaled data effectively. Logistic Regression, however, requires `StandardScaler` for regularization stability.

Model Design Pipeline

- **Baseline**: Logistic Regression. Fast, linear, easily interpretable $\rightarrow$ proves the features have predictive power.
- **Advanced 1**: Random Forest Classifier. Captures non-linear signals.
- **Champion Model**: XGBoost. Handles feature correlation, internal missing data distribution, and optimally splits on density-based criteria. Excellent at optimizing ranking mechanisms.

Validation Strategy

- **Temporal / Time-Based Split**: Standard Train-Test random splits leak data because an order at $T = 2$ might be used to predict $T = 1$.
- Validation will take the *last* known order for every user as the Hold-Out Validation Set, predicting on everything prior.

Prediction Pipeline (Real-Time UI)

1. **Query**: User ID is supplied.
2. **Candidate Generation**: Fetch all distinct products this user has interacted with historically.
3. **Feature Construction**: Hydrate user features, product features, and construct real-time interaction recency.
4. **Prediction**: Run candidate vector through `XGBoost.predict_proba()`.
5. **Heuristic Filter**: Drop predictions < 0.10 threshold. Sort descending.
6. **Return**: Top-K products delivered to Streamlit display.